TEESOULZ -- EXISTING PROJECT TECHNICAL AUDIT

Comprehensive Frontend Architecture, Backend Readiness & Security Assessment | Strict Mode: Inspect Only

Target Domain: teesoulz.vercel.app	**Framework:** Next.js 16.4.2 (App Router)	**Audit Date:** October 2026
Architecture: 100% Client-Side React 19 SPA	**Database / API:** 0% Implemented (Mock Only)	**Audit Mode:** Strict Read-Only Verification

EXECUTIVE SUMMARY & AUDIT VERDICT:
The visual and interactive frontend for **TEESOULZ** is exceptionally complete, featuring **25 distinct client routes**, responsive streetwear layouts, micro-animations (GSAP), and 3D visualizers (Three.js). However, **zero backend infrastructure exists**. There are **no database tables, no server API endpoints, no authenticated sessions, and no active payment gateways**. All state (cart, wishlist, user account, mock checkout) resides solely in browser memory and localStorage.
Verdict: The frontend is 100% production-ready visually and structurally. It can be wired directly to a PostgreSQL database, Prisma ORM, Auth.js, and Razorpay without refactoring the UI layer.

A. Current Technology Stack

Rigorous inspection of package.json, dependencies, build toolchains, and configurations:

Category	Package / Technology	Version	Implementation Finding & Role
Framework	Next.js (App Router)	16.4.2	Implemented. Utilizing React Server Components runtime, though pages use 'use client'.
UI Core	React / React-DOM	19.3.0	Implemented. Latest React 19 release powering client interactivity.
Styling	Tailwind CSS	^4.0.0	Implemented via @tailwindcss/turbopack. Modern CSS-first engine.
Icons	lucide-react	^1.60.0	Implemented across header, footer, product cards, filters, and modals.
Animations	GSAP	^3.15.0	Implemented. High-performance tweening and micro-interactions.
3D Rendering	Three.js & @types/three	^0.186.1	Implemented. Interactive 3D garment/graphics display canvas.
Language	TypeScript	^5.0.0	Strict typing verified across components, context, and data models.
Linting / Build	ESLint 9 / next build	16.4.2	Standard Next.js compiler scripts (dev, build, start, lint).
Backend / ORM	None (Prisma / Mongoose)	N/A	**NOT IMPLEMENTED.** No database driver or schema present.
Auth Service	None (Auth.js / Supabase)	N/A	**NOT IMPLEMENTED.** Purely mocked client context.
Payment SDK	None (Razorpay / Cashfree)	N/A	**NOT IMPLEMENTED.** Simulated checkout with JS setTimeout.

B. Current Architecture & Application Routes

The codebase adheres to the Next.js App Router specification under src/app. **25 distinct routes** were verified:

Route URL	File Path	Classification	Functional Evidence & Status
/	src/app/page.tsx	Implemented	Hero banners, category grids, featured items, recently viewed.
/shop	src/app/shop/page.tsx	UI Only	Full catalog with client filtering (price, category, size, sort).
/product/[slug]	src/app/product/[slug]/page.tsx	UI Only	Dynamic route resolving from static mock data. Add to cart works.
/category/[slug]	src/app/category/[slug]/page.tsx	UI Only	Dynamic category filter based on static slug parameters.
/collections	src/app/collections/page.tsx	UI Only	Seasonal and streetwear drop collection overviews.
/collections/[slug]	src/app/collections/[slug]/page.tsx	UI Only	Collection-specific product listings mapped to static items.
/cart	src/app/cart/page.tsx	Working Client State	Item quantity adjust, remove, subtotal, promo code client mock.
/wishlist	src/app/wishlist/page.tsx	Working Client State	Favorite toggle, move to cart, localStorage persistence.
/checkout	src/app/checkout/page.tsx	UI Only	Address form, pincode check, payment method selector (mocked).
/order-confirmation/[id]	src/app/order-confirmation/...	UI Only	Static receipt display reading generated ID from query/param.
/account	src/app/account/page.tsx	UI Only	Profile, mock order history, saved addresses in localStorage.
/account/login	src/app/account/login/page.tsx	UI Only	Form accepts any input without password/credential check.
/account/register	src/app/account/register/page.tsx	UI Only	Creates fake client user object in StoreContext.
/track-order	src/app/track-order/page.tsx	UI Only	Visual progress bar accepting any order tracking number.
/bestsellers	src/app/bestsellers/page.tsx	UI Only	Filtered list of mock products tagged as bestseller.
/new-in	src/app/new-in/page.tsx	UI Only	Filtered list of mock products tagged as new arrivals.
/sale	src/app/sale/page.tsx	UI Only	Filtered list of items with discount / sale tags.
/size-guide	src/app/size-guide/page.tsx	Implemented	Static size measurement chart for streetwear tees and hoodies.
/about, /contact, /faq	src/app/(static)/page.tsx	Implemented	Brand story, contact form (unwired), accordion FAQ.
/policies/* (4 routes)	src/app/policies/*/page.tsx	Implemented	Legal copy: Privacy, Terms, Shipping, Return & Refund policies.
/api/* (All routes)	None	Not Implemented	Zero API routes. No server handler exists in the project.
/admin/*	None	Not Implemented	Zero administrative routes or backoffice interfaces.

Architectural Pattern Observed: The application runs essentially as a Single Page Application (SPA) on top of Next.js. While the directory layout follows modern App Router standards, nearly all leaf pages declare 'use client' at the top and bind directly to StoreContext.tsx. This means adding Next.js Route Handlers (src/app/api) and Server Actions will not conflict with existing page components.

C. Database and Backend Status

Classification: NOT IMPLEMENTED (Mock Data Only)

- **Source of Truth:** All product data originates from src/data/mockData.ts, containing an in-memory array of **28 hardcoded streetwear products**, 6 category objects, and 4 mock promo codes (WELCOME10, SOUL15, DROP20, FREESHIP).
- **Zero Database Clients:** There is no PostgreSQL, MySQL, MongoDB, Redis, Prisma, or Drizzle schema anywhere in the repository. No migration files exist.
- **Absence of API Handlers:** There is no src/app/api directory. No Next.js route handlers, Server Actions, or backend microservices are implemented. All data manipulation occurs client-side in React state.
- **Persistence Vulnerability:** Cart items, wishlist items, recently viewed IDs, and mock orders rely purely on browser localStorage under keys: teesoulz_cart, teesoulz_wishlist, teesoulz_recent, and teesoulz_user. If a user clears cookies or changes devices, all data disappears.

D. Authentication and Access-Control Status

Classification: UI ONLY / MOCKED (Critical Security Implication)

- **Login & Registration Logic:** Located inside src/app/account/login/page.tsx and src/context/StoreContext.tsx. The login form executes an artificial setTimeout(..., 800) and creates a fake session containing the entered email, a hardcoded name ('Alex Rivers'), and dummy loyalty points (150 pts).
- **No Passwords or Hashes:** Passwords entered in the UI are never hashed, verified against a store, or transmitted to any server. Any password string is accepted.
- **Zero Session Management:** There are no HTTP-only cookies, no JWT verification tokens, and no Next.js middleware guards. Any user can directly navigate to /account by altering local state.
- **Role-Based Access Control (RBAC):** Completely missing. The user type definition (src/types/index.ts) has no role property (e.g., 'ADMIN', 'STAFF', 'CUSTOMER').

E. Product, Inventory, Cart, Wishlist & Order Status

Feature	Status	Implementation Details & Code Evidence
Product Catalog	Mock / UI Only	28 products in src/data/mockData.ts. Includes images, variants (S-XXL), badges (HOT, DROP, LIMITED), tags, and descriptions. Cannot be updated without code deployment.
Inventory Control	Not Implemented	Static boolean flag inStock: true/false on products. No SKU tracking, stock counts, size-specific inventory, or stock locking during checkout.
Shopping Cart	Working Client State	Fully working in React state and localStorage. Supports quantity changes, size/color selection, subtotal calculation, tax calculation (18% GST), and coupon discounts.
Wishlist	Working Client State	Fully functional client-side toggle. Persisted across page refreshes via localStorage. Move to cart helper verified in StoreContext.tsx.
Order Creation	Mock / UI Only	In src/app/checkout/page.tsx, orders are fabricated using Math.random().toString(36) and pushed into the user's localStorage array. No database record is created.

F. Admin Panel Status

Classification: NOT IMPLEMENTED (0% Complete)

- There are no admin pages (e.g., /admin, /dashboard, /admin/products, /admin/orders).
- No UI exists for store managers to add products, adjust pricing, upload photos, manage stock levels, view customer orders, update delivery tracking statuses, or issue refunds.
- Any product change currently requires editing TypeScript code and re-deploying to Vercel.

G. Payment, Shipping & Notification Status

- **Payment Gateways (Razorpay / Cashfree / Stripe): NOT IMPLEMENTED.** In src/app/checkout/page.tsx, selecting 'UPI', 'Card', or 'COD' merely changes a radio button. Clicking 'Place Order' triggers a JavaScript setTimeout of 2000ms that clears the cart and redirects to /order-confirmation without collecting payment or creating a payment intent.
- **Shipping Integration (Shiprocket / Delhivery): NOT IMPLEMENTED.** Pincode check on product and checkout pages evaluates a static regex (/^[1-9][0-9]{5}$/) and displays hardcoded delivery estimates (3-5 days). No logistics API is called.
- **Notifications (Email / SMS / WhatsApp): NOT IMPLEMENTED.** No Resend, SendGrid, Twilio, or WhatsApp Business API integration exists. When an order is placed, zero emails or text messages are dispatched.

H. Missing Environment Configuration

Inspection confirms that no .env, .env.local, or .env.production files are active in the repository. The following **12 environment variables** are required for backend launch:

Category	Variable Identifier	Required Service / Purpose
Database	DATABASE_URL	PostgreSQL connection string (Supabase / Neon / AWS RDS)
Database Pool	DIRECT_URL	Direct connection string for Prisma schema migrations
Authentication	NEXTAUTH_SECRET	Cryptographic salt for JWT session tokens and encryption
Auth URL	NEXTAUTH_URL	Canonical app URL (https://teesoulz.vercel.app)
OAuth (Google)	GOOGLE_CLIENT_ID / _SECRET	One-tap social customer login authentication
Payments	RAZORPAY_KEY_ID	Public client key for Razorpay checkout modal checkout
Payments Secret	RAZORPAY_KEY_SECRET	Private server key to capture payments and sign orders
Payment Webhook	RAZORPAY_WEBHOOK_SECRET	HMAC signature verification for asynchronous webhook events
Media Storage	CLOUDINARY_URL / S3_*	Product image & banner uploads for Admin Panel
Transactional Email	RESEND_API_KEY	Dispatching order confirmations and tracking updates
Logistics API	SHIPROCKET_API_TOKEN	Automated AWB generation and live shipping rate tracking

I. Security Risks & Implementation Blockers

Deploying the current frontend to production as a live commercial store presents severe vulnerabilities:

1. **Client-Authoritative Pricing:** Product prices and cart totals are calculated solely in the browser. A malicious user could edit localStorage or intercepted payloads to purchase products for INR 1.
2. **Zero Authentication Guards:** Protected areas and sensitive order details are not protected by backend JWT/session verification.
3. **Race Conditions & Overselling:** Without ACID database transactions or pessimistic locking, concurrent users could checkout the same limited streetwear drop item.
4. **Fake Checkout Exploit:** An attacker could trigger the checkout completion function to generate fraudulent order IDs and demand order fulfillment.

J. Recommended Architecture (Zero Rebuild)

Do NOT rewrite the existing application. The current Next.js 16 App Router frontend is modern, well-typed, and modular. The recommended architecture augments the existing repo with a seamless full-stack layer:

Layer	Recommended Solution	Justification & Integration Path
Database	PostgreSQL via Neon / Supabase	Serverless connection pooling, instantaneous branching, robust relational integrity.
ORM	Prisma ORM	Type-safe database client matching existing TypeScript definitions (Product, Order, User).
Authentication	Auth.js v5 (NextAuth)	Native Next.js App Router support, secure HTTP-only cookies, OAuth (Google), and credentials hashing (bcrypt).
Payment	Razorpay API + Webhooks	Standard for Indian e-commerce (UPI, Netbanking, Cards, EMI). Secure server-side signature verification.
Admin Panel	Unified src/app/admin/*	Reuses the same Tailwind v4 styling and Lucide icons; protected by Next.js middleware (role === 'ADMIN').
Media Storage	Cloudinary or Uploadthing	Direct signed uploads for high-resolution product imagery and lookbook banners.

K. Phased Implementation Plan

Phase 1: Database Foundation & Product Data Migration (Days 1-3)

- Initialize Prisma: npm install prisma @prisma/client. Configure PostgreSQL connection.
- Define database schemas for User, Product, ProductVariant, Category, Order, OrderItem, Address, and Coupon.
- Write a seed script (prisma/seed.ts) converting the 28 products from mockData.ts into permanent database rows.
- Convert static product routes (/shop, /product/[slug]) to fetch dynamically via Prisma.

Phase 2: Authentication, User Accounts & Server-Side Cart (Days 4-6)

- Implement Auth.js v5 with bcrypt password hashing and Google OAuth provider.
- Replace client mock login with genuine session creation and secure HTTP-only cookies.
- Wire user account dashboard (/account) to real customer profile and order history records.
- Synchronize cart state to database for logged-in users across mobile and desktop.

Phase 3: Real Payment Gateway & Order Pipeline (Days 7-9)

- Integrate official Razorpay Node SDK. Implement /api/checkout/create-order to generate verified payment IDs.
- Enforce server-side pricing calculation so clients cannot manipulate item prices.
- Create /api/webhooks/razorpay route to verify HMAC signatures and transition orders to 'PAID'.
- Trigger automated customer confirmation emails via Resend.

Phase 4: Streetwear Admin Panel & Inventory Manager (Days 10-14)

- Construct protected /admin dashboard with role-based middleware guards.
- Build Product Management: Add, edit, delete streetwear items; size/color matrix; image upload dropzone.
- Build Order Management: Live order status tracking (Pending -> Processing -> Shipped -> Delivered).
- Build Customer & Discount Management: Create flash drop coupons and inspect customer analytics.

L. Final Recommendation & Exact First Implementation Task

EXACT FIRST IMPLEMENTATION TASK TO PROCEED:

Task 1: Initialize Prisma ORM and create the PostgreSQL Database Foundation.

1. Run npm install -D prisma and npm install @prisma/client.
2. Execute npx prisma init to generate the schema structure.
3. Configure the schema.prisma models matching the audited TypeScript types.
4. Execute the initial migration and seed all 28 streetwear items from mockData.ts directly into PostgreSQL.

This establishes the persistent backbone without breaking any existing frontend pages.